

Week 7: SHAP and Feature Importance



MAISI



Building Interpretable AI in Healthcare
Michigan Data Science Team - Winter 2026



Week 7 Agenda

01.

Icebreaker!

Every great project work session starts off with a great icebreaker.

02.

Intro to SHAP Values

What are SHAP Values, and how do they help us interpret model predictions?

03.

Feature Importance

How can we determine which features of a patient contribute most to a prediction, and why is this important for AI safety?

04.

Hands-On Data Science!

Time to get into the notebook and start working through exercises revolving around mechanistic interpretability!

01 Fun Icebreaker!!

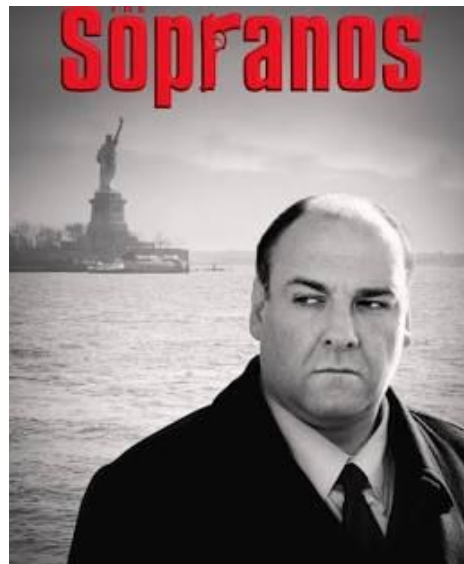
Get to know us and start to learn about
each other with a fun icebreaker!



Icebreaker – Week 7

Share with the people around you!! :)

What is your favorite TV show of all time?





02

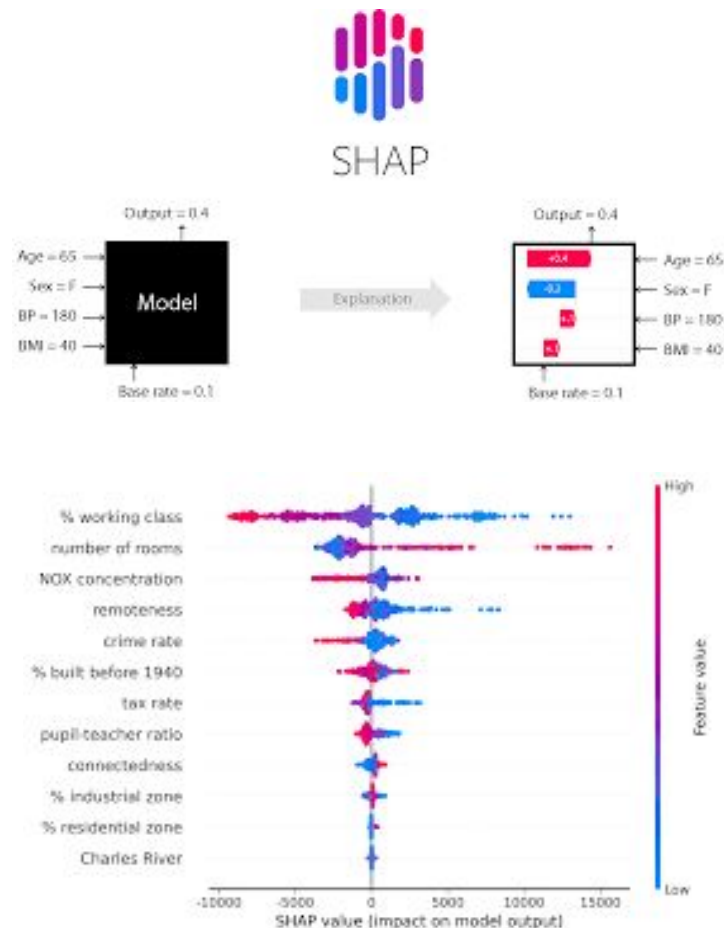
Intro to SHAP Values

What are SHAP Values, and how do they help us interpret model predictions?



What are SHAP Values?

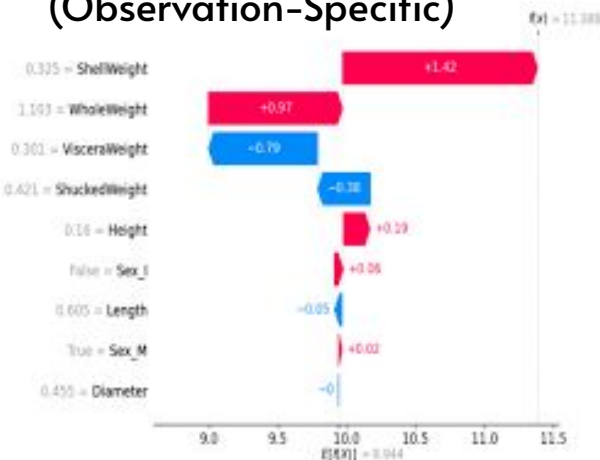
- **Shapley Additive exPlanations (SHAP)** utilize *game theory* to explain how a model got to a decision by quantifying the impact of each feature.
- Can be utilized on the full dataset or on a particular patient.
- **Game Theory aspect:** Calculates the average marginal contribution of each feature across all possible combinations of features of a patient.



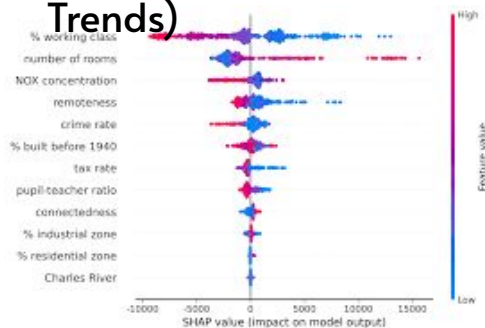
Why are SHAP Values Useful?

- Give us insight into **local** and **global** behavior of any model (model-agnostic), telling us how certain features generally play into predictions.
 - Invaluable insight into the decision-making process of models that are otherwise black boxes!
- Main downside: VERY computationally expensive
 - When a dataset has tons of features, SHAP needs to test tons of different combinations
 - Time Complexity: $O(2^n)$, where n is the number of features

Local (Observation-Specific)



Global (General Trends)





03

Feature Importance

How can we determine which features of a patient contribute most to a prediction, and why is this important for AI safety?



What is Feature Importance?

- **Feature Importance** is a general term that refers to different methods of assigning a score to input features based on how important they are to the predicting a model makes
 - SHAP Values are a wonderful example of a feature importance technique
- Use cases for feature importance techniques include:
 - Feature Selection (EDA process)
 - Mechanistic Interpretability
 - Debugging (Age double-count example from our demo)
 - Business insights (determining what drives desirable outcomes)

What are Examples of Feature Importance?

- **Tree-Based machine learning algorithms** such as Random Forest or XGBoost are often considered feature importance techniques
 - The calculation of scores in these models inherently creates feature importance scores, which can be accessed with `feature_importances_`
- **Regression algorithms** like Linear and Logistic Regression calculate coefficients for each non-target variable included in testing, which can be seen as feature importance scores.
- **Metrics: SHAP, Gini Importance (Trees), Performance Decrease, XGBoost Metrics**

Why is this important for AI Safety?

- **Mechanistic Interpretability:** A field of AI research attempting to work backwards from model outputs to human-understandable, interpretable algorithms.
 - More broadly, this is a growing movement in the AI sector dedicated to making sure that future advancements in AI are safe, equitable, and understandable.
- If we can *understand* a model's rationale for why it makes certain decisions, we can more easily detect biases and prevent the model from learning unsatisfactory lessons.
 - Simply looking at disagreeable outputs can easily be either deceptive or annoying, depending on whether not you catch the bias naturally, but feature importance allows us to do something about it



References

Here are a handful of useful guides to high-utility functions, Git demos, and Colab explanations!

Our Demo! (for reference)



Link:

<https://mdst-ai-in-healthcare.streamlit.app/>

Project Timeline

Week 1 (01/25): Icebreaker/EDA intro, choosing a good dataset

Week 2 (02/01): Data Visualization, Further EDA

Week 3 (02/08): Feature Engineering and Intro to Machine Learning

Week 4 (02/15): Machine Learning and Model Optimization

Week 5 (02/22): Conformal Predictions, Risk Control, and Multi-Class Calibration

NO MEETING 03/01 OR 03/08 – SPRING BREAK

Week 6 (03/15): SHAP Values and Feature Importance Interpretation

Week 7 (03/22): Intro to Streamlit and Project Work Time

Weeks 8 + 9 (03/29 + 04/05): Final Project work time + Data Science Night prep!



Important Functions to Know (pt. 1)

Syntax	Description
<code>import pandas as pd</code>	Import the pandas library, using the alias pd for convenience
<code>import numpy as np</code>	Import the NumPy library, using alias np for numerical operations
<code>import seaborn as sns</code>	Import Seaborn for statistical data visualization
<code>df = pd.read_csv('file.csv')</code>	Load a CSV file into a pandas DataFrame for data manipulation
<code>df.head()</code>	Returns the first 5 rows of the DataFrame, useful for quickly inspecting data
<code>df.info()</code>	Provides a concise summary of the DataFrame, including data types and non-null values

Important Functions to Know (pt. 2)

Syntax	Description
<code>df.describe()</code>	Generates descriptive statistics of numerical columns (mean, median, quartiles, etc.)
<code>df['column_name']</code>	Access a specific column in the DataFrame, works like a key in a dictionary
<code>df.drop(columns=['col'...])</code>	Drops specified columns from the DataFrame, use <code>inplace=True</code> to modify the original DataFrame
<code>df.index</code>	Returns the index labels of the DataFrame
<code>df.isnull()</code>	returns a DataFrame of boolean values , where each entry indicates whether the corresponding value in df is NaN (missing)

Important Functions to Know (pt. 3)

Syntax	Description
<code>df['column'].value_counts()</code>	Returns the count of unique values in a specific column
<code>df['column'].mean()</code>	Returns the mean value of a numerical column
<code>df.corr()</code>	Computes correlation for numerical columns to understand relationships
<code>df.columns</code>	Lists all column names in the DataFrame , useful for renaming or viewing dataset structure
<code>df.shape</code>	Returns # of rows and columns

Important Functions to Know (pt. 4)

Syntax	Description
<code>df.rename(columns={'old' : 'new'}, inplace=True)</code>	Renames specific columns to new names
<code>df.groupby('category')['value']</code>	Groups the DataFrame using a specified column to perform aggregate functions (e.g., <code>.sum()</code> , <code>.mean()</code>)
<code>df['new_column'] = df['column'].apply(function)</code>	Applies a function to each element or column/row
<code>df.shape</code>	Returns a tuple representing the dimensions (rows, columns) of the DataFrame
<code>df.dropna()</code>	Removes rows or columns containing missing values

Important Functions to Know (pt. 5)

Syntax	Description
<code>df.to_numpy()</code>	Converts Pandas DataFrame into a NumPy array
<code>np.shape</code>	Returns a tuple representing the dimensions (rows, columns) of the array (similar to <code>df.shape</code>)
<code>np.reshape()</code>	Reshapes an original numpy array to the specified dimensions (rows, columns)
<code>np.array()</code>	Creates an N-dimensional array (ndarray) which is more memory efficient than standard python lists
<code>np.unique()</code>	Returns the unique elements in a NumPy array (also returns the counts of each element given the keyword argument, <code>return_counts=True</code>)

Important Functions to Know (pt. 6)

Syntax	Description
<code>pd.cut()</code>	Turns continuous numerical data into categorical data
<code>pd.get_dummies(drop_first = True)</code>	Dummy encodes categorical data (used with <code>drop_first</code> argument)
<code>OrdinalEncoder()</code>	Function for encoding categorical information that has an order
<code>train_test_split(X, y, test_size=0.2, random_state=42)</code>	Splits data into a training set and a testing set . The test set size can be changed
<code>LinearRegression()</code>	Used for initializing a linear regression model
<code>LogisticRegression()</code>	Used for initializing a logistic regression model

Important Functions to Know (pt. 7)

Syntax	Description
StandardScaler()	Turns continuous numerical data into categorical data
model.fit(X_train, y_train)	Fits a model to the training data
model.predict(X_test)	Uses the test data to make predictions
.coef_	Attribute for getting the models coefficients (for linear and logistic regression)
mean_squared_error()	Calculates the average squared distance between the predicted and actual data points
mean_absolute_error()	Calculates the average distance between the predicted and actual values

How to Upload Files into Colab

Click on the folder in the sidebar



The screenshot shows a Google Colab notebook interface. On the left, the 'Files' sidebar is open, showing a folder named 'sample_data'. A red arrow points to this folder from the text 'Click on the folder in the sidebar'. The notebook content area has a title 'Week 1 - Pandas Practice'. Below the title, there is a text prompt: 'Here is where you import the libraries necessary to perform the following tasks!'. The first code cell contains the following code:

```
import pandas as pd
import seaborn as sns

# Allows you to provide a path to a Google Drive address rather than a local file path
from google.colab import drive
drive.mount('/content/drive')
```

Below the code cell, it says 'Mounted at /content/drive'. The second text prompt says: 'Load the Google Forms .csv into a Pandas dataframe.'. The second code cell contains the following code:

```
df = pd.read_csv('/content/MDST Week 1 - Pandas Practice.csv')
```

The third text prompt says: 'Print out the .head() and the datatypes.'. The third code cell contains the following code:

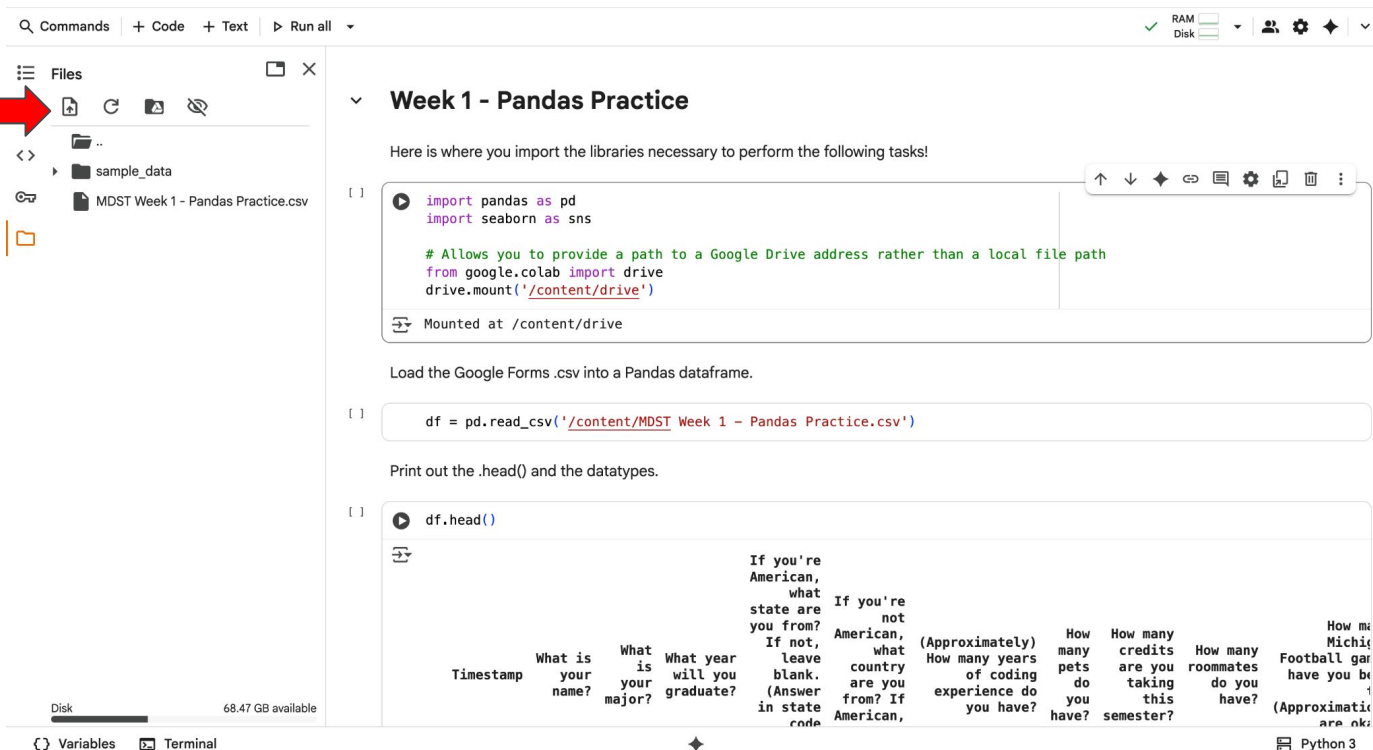
```
df.head()
```

Below the code cell, the output of the code is displayed as a table with 10 columns. The columns are: 'Timestamp', 'What is your name?', 'What is your major?', 'What year will you graduate?', 'If you're American, what state are you from?', 'If not, leave blank. (Answer in state code)', 'If you're American, what country are you from? If American, (Approximately) How many years of coding experience do you have?', 'How many pets do you have?', 'How many credits are you taking this semester?', and 'How many roommates do you have?'. The output shows the first few rows of data.

At the bottom of the notebook, there is a status bar showing 'Disk' with a progress bar and '68.47 GB available'. On the right, it says 'Python 3'.

How to Upload Files into Colab

Click the upload button and select the file you want to upload



The screenshot shows the Google Colab interface. On the left, the 'Files' pane displays a directory structure with a folder named 'sample_data' and a file named 'MDST Week 1 - Pandas Practice.csv'. A red arrow points to the upload button (a square with a plus sign) in the top left of the Files pane. The main area is titled 'Week 1 - Pandas Practice' and contains the following text: 'Here is where you import the libraries necessary to perform the following tasks!'. Below this, a code cell contains the following Python code:

```
import pandas as pd
import seaborn as sns

# Allows you to provide a path to a Google Drive address rather than a local file path
from google.colab import drive
drive.mount('/content/drive')
```

The output of the code cell shows 'Mounted at /content/drive'. Below the code cell, the text 'Load the Google Forms .csv into a Pandas dataframe.' is followed by a code cell containing:

```
df = pd.read_csv('/content/MDST Week 1 - Pandas Practice.csv')
```

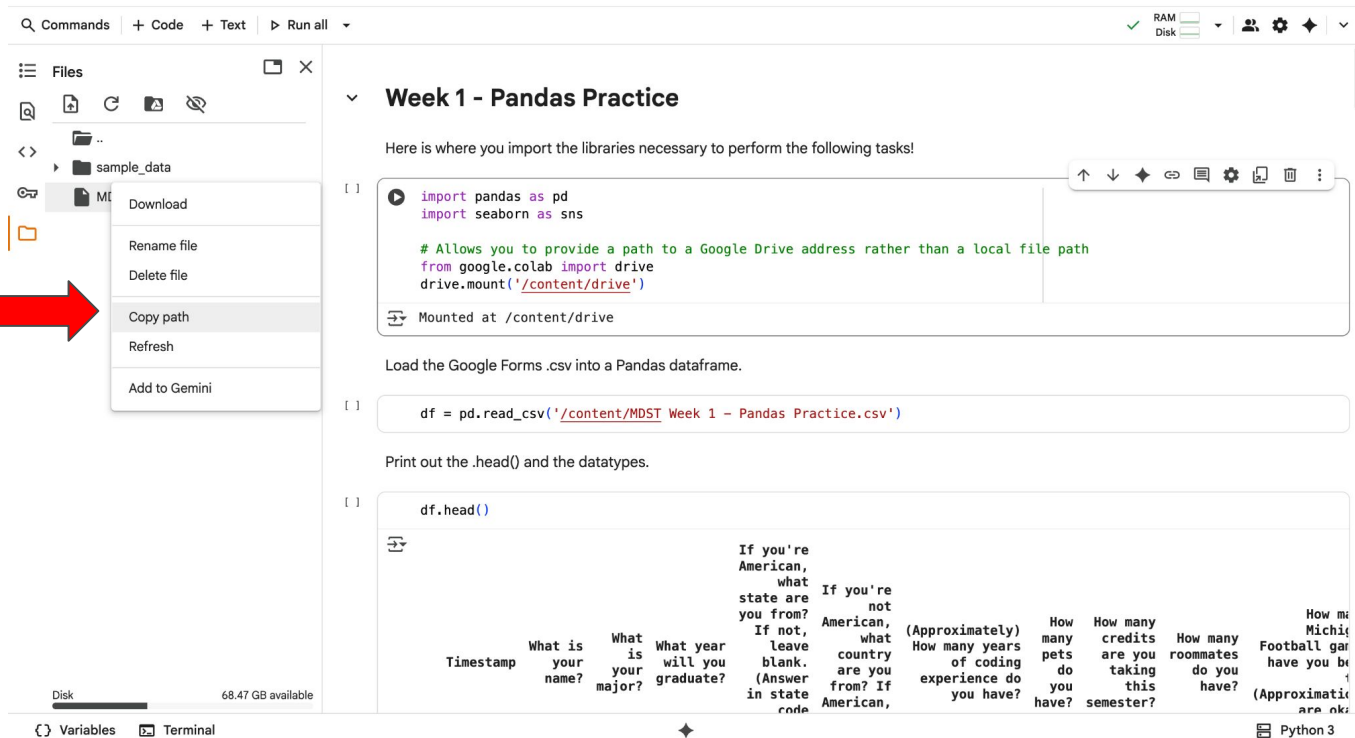
Below this, the text 'Print out the .head() and the datatypes.' is followed by a code cell containing:

```
df.head()
```

The output of the code cell shows the first five rows of the dataset, which includes columns for 'Timestamp', 'What is your name?', 'What is your major?', 'What year will you graduate?', 'If you're American, what state are you from?', 'If you're not American, what country are you from?', 'If not, leave blank. (Answer in state code)', 'How many years of coding experience do you have?', 'How many pets do you have?', 'How many credits are you taking this semester?', 'How many roommates do you have?', and 'How many football games have you been to? (Approximate)'. The output is displayed in a table format.

How to Upload Files into Colab

Click the three dots and copy the path. Put this in your read function



The screenshot shows the Google Colab interface. On the left, the 'Files' pane displays a folder named 'sample_data' containing a file 'MDST Week 1 - Pandas Practice.csv'. A context menu is open for this file, with the 'Copy path' option highlighted. A red arrow points from the text 'Click the three dots and copy the path. Put this in your read function' to this option. The main editor area is titled 'Week 1 - Pandas Practice' and contains the following text and code:

Here is where you import the libraries necessary to perform the following tasks!

```
[ ] import pandas as pd
import seaborn as sns

# Allows you to provide a path to a Google Drive address rather than a local file path
from google.colab import drive
drive.mount('/content/drive')
```

Mounted at /content/drive

Load the Google Forms .csv into a Pandas dataframe.

```
[ ] df = pd.read_csv('/content/MDST Week 1 - Pandas Practice.csv')
```

Print out the .head() and the datatypes.

```
[ ] df.head()
```

The output of the code shows the first few rows of the CSV file, which contains survey data about American citizens. The columns include 'Timestamp', 'What is your name?', 'What is your major?', 'What year will you graduate?', 'If you're American, what state are you from?', 'If not, what country are you from?', 'How many years of coding experience do you have?', 'How many pets do you have?', 'How many credits are you taking this semester?', 'How many roommates do you have?', and 'How many football games have you been to?'. The data is displayed in a table format.

How to Mount Your Google Drive

```
from google.colab import drive

drive.mount('/content/drive')

pd.read_csv('/content/drive/MyDrive/[FILE NAME]')
```

****NOTE:** If you saved your dataset in a folder (not just loose in your MyDrive folder), you will need to add further code to the file path before the file name in the final line. For example, if I saved my data called “alzheimers.csv” in a folder called “MDST-W26,” my read_csv statement would read:

```
pd.read_csv('/content/drive/MyDrive/MDST-W26/alzheimers.csv')
```